

Kubernetes & Amazon EKS

An Introduction + Labs from Basic to Advanced

Understand what Kubernetes and EKS do, how the pieces fit together, and practice with six hands-on labs on a real EKS cluster.

Part 1 — What Is Kubernetes?

Kubernetes (often shortened to "K8s") is an open-source system for running and managing containers — like Docker containers — across many machines, automatically. If Docker packages an application into a container, Kubernetes is the layer that decides where that container runs, keeps it running, restarts it if it crashes, and scales it up or down based on demand.

Without Kubernetes, if you're running containers across multiple servers, you'd have to manually decide which server runs which container, watch for crashes, and reroute traffic by hand. Kubernetes automates all of that.

What Problems Does Kubernetes Solve?

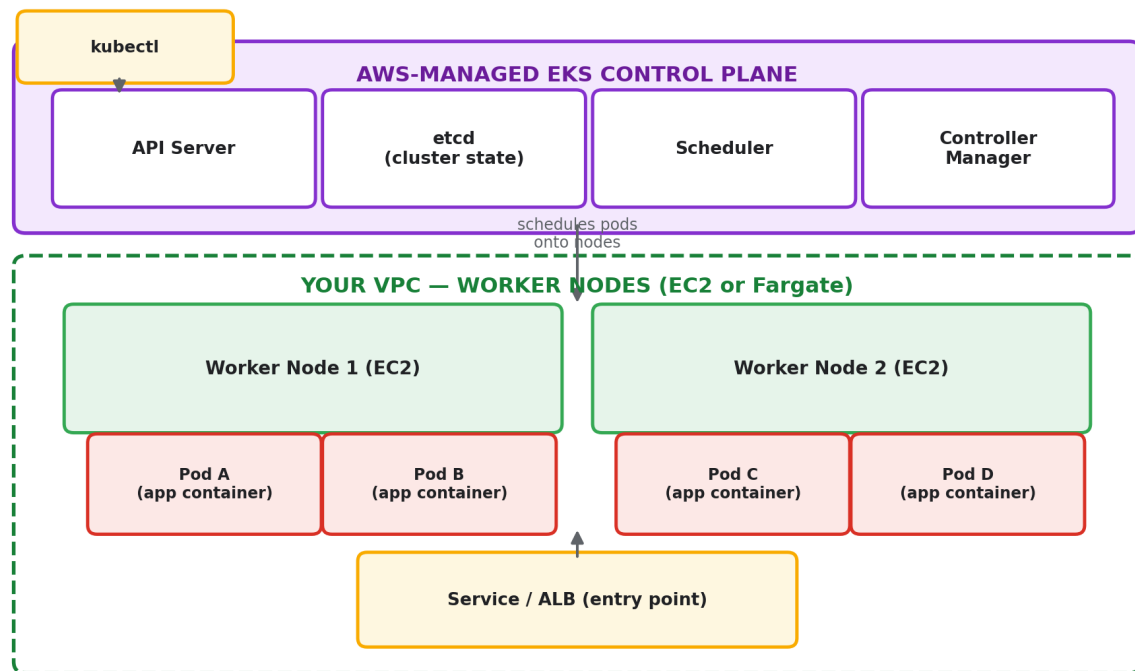
- Scheduling — automatically decides which server (node) runs each container based on available resources.
- Self-healing — restarts containers that crash, replaces unhealthy ones, and reschedules workloads if a node fails.
- Scaling — increases or decreases the number of running copies of an app based on load.
- Service discovery & load balancing — gives containers stable network identities and spreads traffic across replicas.
- Rolling updates & rollbacks — deploys new versions gradually, and can automatically revert if something breaks.
- Declarative configuration — you describe the desired end state (e.g. "run 3 copies of this app") in a YAML file, and Kubernetes continuously works to match reality to that description.

Key Kubernetes Concepts

- Cluster — the whole Kubernetes environment: a control plane plus a set of worker machines (nodes).
- Node — a machine (VM or physical server) that runs your containers.
- Pod — the smallest deployable unit in Kubernetes; wraps one or more tightly-coupled containers that share networking and storage.
- Deployment — describes how many copies (replicas) of a Pod should run, and manages rolling updates.
- Service — a stable network endpoint that load-balances traffic across a set of Pods, even as individual Pods are replaced.
- Namespace — a way to logically divide a cluster (e.g. dev, staging, prod) within the same physical infrastructure.
- ConfigMap / Secret — externalized configuration and sensitive values injected into Pods without baking them into the image.

Part 2 — What Is Amazon EKS, and How Does It Work?

Amazon Elastic Kubernetes Service (EKS) is AWS's managed Kubernetes offering. Running Kubernetes yourself means operating the control plane — the API server, etcd database, scheduler, and controller manager — which is complex and failure-sensitive. EKS takes that burden off you: AWS runs and maintains a highly available control plane, patches and upgrades it, and exposes it to you through the standard Kubernetes API. You focus on your applications and worker nodes; AWS focuses on keeping the cluster's brain alive.



How the Pieces Fit Together

1. You interact with the cluster using `kubectl`, the Kubernetes command-line tool, which talks to the API Server.
2. The API Server is part of the AWS-managed control plane — you never see or manage the underlying servers running it.
3. The Scheduler decides which worker node should run each new Pod, based on available CPU/memory and any placement rules.
4. Worker nodes run inside your own VPC — either as EC2 instances you manage (a "managed node group"), or as AWS Fargate, where AWS runs each Pod without you provisioning any EC2 instances at all.
5. A Service or an Application Load Balancer (via the AWS Load Balancer Controller) exposes your Pods to the internet or to other AWS services.

EKS vs. Self-Managed Kubernetes vs. ECS

Self-Managed Kubernetes	Amazon EKS
You install, patch, and scale the control plane yourself	AWS runs a highly available, patched control plane for

Self-Managed Kubernetes	Amazon EKS
	you
Full flexibility, full operational burden	Same Kubernetes API, far less operational overhead
Common on bare EC2, on-prem, or other clouds	Deep integration with IAM, VPC, ALB, CloudWatch, ECR

EKS is not the same as Amazon ECS (Elastic Container Service). ECS is AWS's own simpler, proprietary container orchestrator; EKS runs the open-source Kubernetes API, which is useful if you want portability across clouds or already have Kubernetes expertise/tooling.

Part 3 — Before the Labs: One-Time Setup

These labs use eksctl, AWS's official CLI tool for creating and managing EKS clusters, which handles the underlying VPC, IAM roles, and node groups for you in one command.

6. Install the AWS CLI, kubectl, and eksctl on your machine:

```
brew install awscli kubectl eksctl
aws configure          # enter your Access Key, Secret Key, and region
```

7. Confirm everything is installed:

```
aws --version
kubectl version --client
eksctl version
```

Cost note: an EKS cluster's control plane costs about \$0.10/hour, plus the cost of any EC2 worker nodes. Delete the cluster at the end of the labs to avoid ongoing charges.

Basic Labs

Lab 1 — Create Your First EKS Cluster

BASIC

Goal: stand up a real EKS cluster with worker nodes using a single command.

8. Create a cluster with two t3.medium worker nodes (takes ~15-20 minutes):

```
eksctl create cluster \
  --name my-first-cluster \
  --region us-east-1 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed
```

eksctl provisions the EKS control plane, a VPC, subnets, and an EC2 managed node group, and automatically configures kubectl to talk to the new cluster.

9. Confirm the cluster and nodes are ready:

```
kubectl get nodes
```

You should see two nodes in Ready status. You can also see the cluster listed under EKS in the AWS Console.

Lab 2 — Deploy and Expose Your First App

BASIC

Goal: run a simple app as a Deployment, expose it with a Service, and scale it.

10. Create a Deployment running Nginx:

```
kubectl create deployment hello-nginx --image=nginx:alpine --replicas=2
```

11. Check the Pods it created:

```
kubectl get pods
```

12. Expose it internally with a ClusterIP service:

```
kubectl expose deployment hello-nginx --port=80 --type=ClusterIP
```

13. Now expose it to the internet with a LoadBalancer service (EKS provisions a real AWS ELB):

```
kubectl expose deployment hello-nginx --port=80 --type=LoadBalancer --
name=hello-nginx-public
```

14. Get the public address (may take a minute or two to populate):

```
kubectl get service hello-nginx-public
```

Open the EXTERNAL-IP shown (a DNS hostname) in your browser to see the Nginx welcome page, now served by pods running inside your EKS cluster.

15. Scale the app up:

```
kubectl scale deployment hello-nginx --replicas=4  
kubectl get pods
```

Moderate Labs

Lab 3 — Configuration with ConfigMaps and Secrets

MODERATE

Goal: externalize configuration and sensitive values instead of hardcoding them into your app or image.

16. Create a ConfigMap with a couple of settings:

```
kubectl create configmap app-config --from-literal=APP_MODE=production --from-literal=APP_COLOR=blue
```

17. Create a Secret for a sensitive value:

```
kubectl create secret generic app-secret --from-literal=API_KEY=super-secret-value
```

18. Create a Pod YAML that consumes both as environment variables:

```
cat > configured-pod.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: configured-app
spec:
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "env | grep APP; env | grep API; sleep 3600"]
      envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: app-secret
EOF
kubectl apply -f configured-pod.yaml
```

19. Check the logs to see the values injected at runtime, not baked into any image:

```
kubectl logs configured-app
```

Lab 4 — Rolling Updates, Rollbacks, and Health Probes

MODERATE

Goal: safely roll out a new version of an app, and automatically roll back if it's broken.

20. Add readiness and liveness probes to your existing Deployment:

```
kubectl set probe deployment/hello-nginx --readiness --get-url=http://:80/ --initial-delay-seconds=5
kubectl set probe deployment/hello-nginx --liveness --get-url=http://:80/ --initial-delay-seconds=10
```

21. Trigger a rolling update to a new image version:

```
kubectl set image deployment/hello-nginx nginx=nginx:1.27-alpine
kubectl rollout status deployment/hello-nginx
```

Kubernetes replaces Pods gradually, only routing traffic to new Pods once their readiness probe passes — old Pods stay up until replacements are healthy.

22. View rollout history:

```
kubectl rollout history deployment/hello-nginx
```

23. Roll back to the previous version if something goes wrong:

```
kubectl rollout undo deployment/hello-nginx
```


Advanced Labs

Lab 5 — Autoscaling with the Horizontal Pod Autoscaler

ADVANCED

Goal: automatically scale Pod replicas up and down based on real CPU load.

24. Install the metrics-server (required for autoscaling to read CPU/memory usage):

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
```

25. Set a CPU request on your Deployment (HPA needs this to calculate percentage usage):

```
kubectl set resources deployment hello-nginx --requests=cpu=100m
```

26. Create a Horizontal Pod Autoscaler targeting 50% CPU usage, between 2 and 6 replicas:

```
kubectl autoscale deployment hello-nginx --cpu-percent=50 --min=2 --max=6
```

27. Generate load to watch it scale (in a separate terminal, run a busy-loop client):

```
kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c \
  "while true; do wget -q -O- http://hello-nginx.default.svc.cluster.local;
done"
```

28. Watch replicas increase as CPU load rises:

```
kubectl get hpa -w
```

29. Clean up the load generator when done:

```
kubectl delete pod load-generator
```

Lab 6 — IAM Roles for Service Accounts (IRSA): Let a Pod Access AWS Services Securely

ADVANCED

Goal: grant a single Pod fine-grained AWS permissions (e.g. read from an S3 bucket) without putting AWS keys anywhere in your code or cluster — a core EKS security pattern.

30. Associate an IAM OIDC provider with your cluster (one-time per cluster):

```
eksctl utils associate-iam-oidc-provider --cluster my-first-cluster --approve
```

31. Create an IAM service account bound to an S3 read-only policy:

```
eksctl create iamserviceaccount \
  --cluster my-first-cluster \
  --namespace default \
  --name s3-reader \
  --attach-policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess \
  --approve
```

This creates a Kubernetes ServiceAccount that's invisibly linked to a real IAM role — only Pods using this ServiceAccount can assume those permissions.

32. Run a Pod using that service account and confirm it can reach S3 without any hardcoded credentials:

```
cat > s3-test-pod.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: s3-test
spec:
  serviceAccountName: s3-reader
  containers:
    - name: aws-cli
      image: amazon/aws-cli
      command: ["sh", "-c", "aws s3 ls && sleep 3600"]
EOF
kubectl apply -f s3-test-pod.yaml
kubectl logs s3-test
```

The command succeeds because the Pod's IAM role — not any embedded secret — grants it S3 access, scoped only to what that specific ServiceAccount was given.

Clean-Up

EKS clusters incur ongoing cost even when idle, so tear everything down once you're finished:

33. Delete any load balancer services first (so their AWS ELBs are released):

```
kubectl delete service hello-nginx-public
```

34. Delete the whole cluster (removes the control plane, node group, and associated VPC resources):

```
eksctl delete cluster --name my-first-cluster --region us-east-1
```

35. Confirm in the AWS Console under EKS that the cluster is gone, and under EC2 that no worker node instances remain.

Summary

Kubernetes automates the scheduling, healing, scaling, and networking of containers across a fleet of machines, so you describe desired state rather than manage servers by hand. Amazon EKS removes the burden of running Kubernetes' own control plane, letting you focus on worker nodes and applications while AWS keeps the cluster's brain highly available and integrated with IAM, VPC, and load balancing. Across these six labs, you went from creating a cluster and exposing a first app, through externalized configuration and safe rolling updates, up to real autoscaling and secure AWS access from inside a Pod — the same progression used to operate production workloads on EKS.